

I am starting Lab 3 for CS254 (Neural Networks in PyTorch). I have a solid grasp of the underlying conceptual fundamentals, but I am less experienced with PyTorch syntax, tensor operations, and standard boilerplate structures.

To comply with Ashesi University's academic integrity policies, you must act as a strict technical reviewer and pedagogical guide, NOT a code solver:

Do not write full code blocks or solve TODO sections for me.

When I share a TODO or a error traceback, explain the underlying PyTorch paradigm (e.g., tensor shapes, gradient tracking, autograd mechanics) or provide minimal, abstract syntax snippets (pseudo-code or simple array examples) rather than solving my lab task directly.

If I ask for syntax guidance, guide me using PyTorch documentation patterns (e.g., `nn.Module`, `.zero_grad()`, `loss.backward()`, `optimizer.step()`) so I can write the logic myself.

Keep track of our debugging sessions so I can accurately declare our interaction on my AI Use Declaration Form at the end.

Acknowledge these guardrails, and I will share the first TODO task.



+ Ask Gemini

Flash ▾



Gemini is AI and can make mistakes.

My next few todos!

...

TODO: Initialise w and b to small random values.

w = 0.2

b = 0.1

TODO: Implement ONE gradient-descent step by hand,
then loop it for ~200 steps:

lr = 0.01 # learning rate!

y_hat = w * x + b

loss = np.mean((y_hat - y) ** 2) # MSE

dw = np.mean(2 * (y_hat - y) * x) # dLoss/dw (derive
this on paper!)

db = np.mean(2 * (y_hat - y)) # dLoss/db

w -= lr * dw

b -= lr * db

print(loss, dw, db, w, b)

Record the loss at every step.

descent_data = []

iters = 0

while iters <= 200 and loss > 0.05:

 y_hat = w * x + b

 loss = np.mean((y_hat - y) ** 2) # MSE

 dw = np.mean(2 * (y_hat - y) * x) # dLoss/dw (derive
this on paper!)

 db = np.mean(2 * (y_hat - y)) # dLoss/db

 w -= lr * dw

 b -= lr * db

 descent_data.append([loss, dw, db, w, b])

 iters += 1

...

+ Ask Gemini

Flash ▾



Gemini is AI and can make mistakes.

Conceptual Check: Derivatives & Optimization Flow

Your manual calculus derivations for the gradients are spot on. Let's trace what your code is doing mathematically to make sure you see how the loop structure aligns with standard training paradigms.

1. The Calculus Verification

For a linear model $\hat{y}_i = wx_i + b$ and Mean Squared Error loss:

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Using the chain rule with respect to w :

$$\frac{\partial L}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2(\hat{y}_i - y_i) \cdot \frac{\partial \hat{y}_i}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2(\hat{y}_i - y_i)x_i$$

Your code implementation `np.mean(2 * (y_hat - y) * x)` directly maps to this vector mean. The bias derivative follow the same logic ($\frac{\partial \hat{y}_i}{\partial b} = 1$).

Loop Structure & Redundancy

Notice that before your `while` loop, you executed a single step:



Gemini is AI and can make mistakes.

I think I understand your gripe about implementing a first step before looping, but take a look at the todo:
> # TODO: Implement ONE gradient-descent step by hand, then loop it for ~200 steps:

It asks me to compute one step and then loop it for about 200 times. I understand that it feels a bit odd, but I was just following instructions. Or? Do you still advise against it? Secondly, I agree that a linear $w x + b$ trying to approximate a non-linear function is odd, but I think the lab is just trying to set me up for the other concepts that will come through soon

Fair enough — if the lab explicitly asks for "one step by hand, then loop," following the prompt's layout makes total sense for the assignment structure.

The main thing to keep in mind for your `descent_data` tracking is just consistency: because `w` and `b` get updated in that solo first step, the weights entering step 1 of your loop are already updated once. As long as you are aware of that offset when plotting your loss curve later, you're fine.

And you hit the nail on the head regarding the model capacity: using a linear model on $\sin(x)$ first is a classic pedagogical tool. It shows you the limits of single-layer linear models before introducing non-linear activation functions and multi-layer architectures later in the lab.

Debugging Log Status:

- **Task:** TODO #2 & #3 — Manual Gradient Descent Steps
- **Concepts Covered:** Lab structure compliance, tracking step offsets in manual loops, motivation for linear baseline on non-linear targets.

+ Ask Gemini

Flash ▾ 🎤

Gemini is AI and can make mistakes.

The next section is on 1.2. We will be building with more neurons (or layers/depth). Currently, the first TODO is to initialise parameters that we will work with. Here are the block comments:

```
---
...

# TODO: Initialise parameters
# W1: shape (1, 8), b1: shape (8)
# W2: shape (8, 1), b2: shape (1,)
# Small random values (e.g. np.random.randn(...)* 0.5)
...
```

There is a bit that I do not yet understand wrt the initialisation. Are we initialising one weight for each neuron or layer? And how does one homogenous weight for layer even work? Conceptually, I have always known learning to need many features or parameters. Even something as simple as linear regression needs many weights for each "feature" or "dimension". So is there a reason why we want to have one weight for each neuron? Or have I misunderstood? Secondly, it seems the shape is mighty important. In fact, it almost looks like the shape for a bias is the transposed shape of the shape of a weight (for both w1 and w2). Is this a numpy quirk where the dot product is easily computed with the transpose (allowing for final addition with b)?



Weight Initialisation & Matrix Shapes

Your intuition about needing multiple parameters per neuron is right on the mark—the key lies in how the inputs, weights, and biases are aligned mathematically in matrix multiplication.

+ Ask Gemini

Flash



Gemini is AI and can make mistakes.

1. Are we initialising one weight per neuron?

No, every neuron gets a weight for *each* of its inputs.

In a fully connected (dense) layer with I inputs and O outputs (neurons):

- Every single neuron j (out of O) receives connections from **all** I inputs.
- Therefore, **1 neuron needs I weights**.
- A layer with O neurons needs $I \times O$ total weights.

Let's look at your layer dimensions:

- **Layer 1 (W_1): Shape (1, 8)**
 - **Input size (I_1):** 1 (our 1D scalar feature, x).
 - **Output size (O_1):** 8 (we have 8 neurons in this hidden layer).
 - **Parameters:** Each of the 8 hidden neurons takes 1 input. So neuron 1 has 1 weight, neuron 2 has 1 weight, ..., neuron 8 has 1 weight. Total = $1 \times 8 = 8$ weights.
- **Layer 2 (W_2): Shape (8, 1)**
 - **Input size (I_2):** 8 (the 8 activation outputs coming from Layer 1).
 - **Output size (O_2):** 1 (the single scalar prediction, $\hat{y}$).
 - **Parameters:** The 1 output neuron receives inputs from all 8 hidden neurons. So that single neuron needs **8 weights** (one for each hidden neuron's output). Total = $8 \times 1 = 8$ weights.

So a single weight is **not** representing an entire layer. Each weight represents a specific edge (synapse) connecting one input node to one output node.

2. Matrix Multiplication & Bias Shapes

You noticed that b_1 has shape (8,) while W_1 is (1, 8). This is driven by **linear algebra** shape alignment and NumPy's broadcasting rules, not transpose rules.

+ Ask Gemini

Flash ▾ 🔊

This is a good place to then try and make sense of how or why backpropagation works. From a residual understanding of the concept, I think that backprop involves computing the loss at every level until we get to the input and then adjusting the value so that the next pass forward is closer to an ideal model weight. But how? But why? Why must we chain it together? Why does our earlier model compute the gradient at a level simply and alter instead of chaining through individual layers? Furthermore, is there even a closed form solution? Why/Why not?



1. Why Do We Chain It Together? (The Chain Rule)

In a single-neuron model ($y = wx + b$), the input x connects directly to the output. The loss depends on w without any intermediate steps, so finding $\frac{\partial L}{\partial w}$ is a straightforward single derivative.

In a multi-layer network, weights in earlier layers (like W_1) do not touch the final output $\hat{y}$ directly. Instead, they influence z_1 , which influences h , which influences $\hat{y}$, which finally influences the scalar L :

$$W_1 \longrightarrow z_1 \longrightarrow h \longrightarrow \hat{y} \longrightarrow L$$

To know how a small change in W_1 affects L , calculus dictates that you must multiply the rates of change along every step of that chain:

$$\frac{\partial L}{\partial z_1} \frac{\partial h}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial L} \frac{\partial L}{\partial W_1}$$

+ Ask Gemini

Flash ▾



Gemini is AI and can make mistakes.

Layer 1 in isolation. Layer 1's performance can only be evaluated by running its outputs through Layer 2, seeing how far off the final prediction is, and sending that error signal backward.

4. Is There a Closed-Form Solution? Why or Why Not?

No, there is no closed-form solution for multi-layer neural networks with non-linear activation functions.

Why Linear Regression Has a Closed-Form Solution

In linear models ($Y = XW$), the loss function is convex (quadratic). Setting the gradient to zero yields a direct matrix formula (the Normal Equation):

$$W = (X^T X)^{-1} X^T Y$$

Why Deep Networks Do Not

1. **Non-linear Activations:** Functions like tanh, ReLU, or sigmoid interlock parameters in non-linear transcendental equations. You cannot algebraically isolate W_1 in $Y = \tanh(XW_1)W_2$ to solve for $\nabla L = 0$.
2. **Non-Convex Loss Landscapes:** Combining multiple layers with non-linearities creates complex, non-convex loss surfaces filled with saddle points, plateaus, and local minima.
3. **Symmetry Breaking:** Because all hidden neurons start as interchangeable units, there are infinitely many combinations of weights that yield the exact same output function (permutation symmetry).

Because no analytical formula can isolate the optimal weights, we rely on **iterative optimization** (gradient descent) to incrementally walk down the loss landscape step-by-step.

Debugging Log Status:

+ Ask Gemini

Flash ▾ 🔊

Gemini is AI and can make mistakes.

I seem to be getting this error:

```
/tmp/ipykernel_328340/3715987062.py:22: RuntimeWarning:
overflow encountered in square
loss = np.mean((y_hat - y) **
2)/tmp/ipykernel_328340/3715987062.py:30:
RuntimeWarning: overflow encountered in matmul
dh = d_yhat @ W2.T/tmp/ipykernel_328340/3715987062.py:31:
RuntimeWarning: invalid value encountered in multiply
dz1 = dh * (1 - np.tanh(z1) ** 2) # derivative of
tanh/home/onkhida/Documents/Ashesi/CS254/labs/Eta_Dani
el_Lab3/venv/lib/python3.13/site-
packages/matplotlib/scale.py:270: RuntimeWarning: overflow
encountered in power
return np.power(self.base, values)
```

I assume this exploding gradient is making some extreme values. Normally, I would reduce the learning rate, but it was given to me by the lab. As such, how do I handle this? Or clip it, idk?

You hit the nail on the head: this is a classic **exploding gradient** and numerical overflow, triggered specifically by `lr = 1.0`.

Why is `lr = 1.0` Exploding?

When the learning rate is too high (like `1.0`), parameter updates take giant leaps across the loss landscape instead of small, controlled steps:

+ Ask Gemini

Flash ▾ 

Gemini is AI and can make mistakes.

I need a re-review of what `nn.Linear` does + what each neuron is meant to do? Is one neuron for one feature etc.? Heuristics for knowing the architecture after seeing a line of code or knowing how to compute how many layers and neurons there are? Still a bit mystic for me

What `nn.Linear` Does at the Scalar Level

A single `nn.Linear` layer represents a **fully connected (dense) affine transformation**:

$$\text{Out} = XW^T + b$$

To build an intuitive mental model, break it down from a single neuron up to the layer level:

1. What is ONE neuron doing?

- It does **NOT** represent a **single feature**. Every neuron in a linear layer looks at **ALL features** coming from the previous layer simultaneously.
- If your input vector has 10 features ($x_1, x_2, \dots, x_{10}$), a single neuron holds 10 weights ($w_1, w_2, \dots, w_{10}$) and 1 scalar bias (b).
- That single neuron calculates a single weighted sum across all 10 features:

$$\text{Neuron Output} = (w_1x_1 + w_2x_2 + \dots + w_{10}x_{10}) + b$$

- **Intuition:** A single neuron is a "feature combiner" or a "pattern detector." It takes all input

+ Ask Gemini

Flash ▾ 🗣️

Gemini is AI and can make mistakes.

So how does this differ from parameter count? Is a feature a parameter count?

Here is the distinction: **features and parameters are fundamentally different concepts.**

1. What is a Feature vs. a Parameter?

- **Features (X):** The **inputs** or raw data values fed into the model. They are given to the network, not learned.
 - *Example:* In your obesity dataset, **Age**, **Height**, **Weight**, and **Gender** are individual **features**. If your dataset has 18 preprocessed columns, you have **18 features** (`input_dim = 18`).
- **Parameters (W and b):** The internal **learnable weights and biases** that the network adjusts during training via backpropagation.
 - *Example:* The values in **W_1** and **b_1** are **parameters**.

2. Is a Feature a Parameter Count?

No. A feature is an input variable, while a parameter is a learnable number inside a layer.

However, the **number of features determines how many parameters a layer needs to process those features.**

The Relationship Formula

+ Ask Gemini

Flash ▾



Gemini is AI and can make mistakes.

Ummm, but still not sure how I will access that in the final block. That is, the model that had the highest accuracy, how do I get its time?

Here's my take: You can store the winning model's training time in your `global_tracker` dictionary right alongside the best validation accuracy!

Since `global_tracker` already updates whenever a new best accuracy is achieved, you can record the elapsed time for that run in the exact same `if` statement.

Step 1: Update `run_experiment` to Track Time on Best Models

Update your `run_experiment` function to measure training duration and save it into `global_tracker`:

```
Python

import time

def run_experiment(
    hidden_sizes=(64, 32),
    lr=1e-3,
    batch_size=32,
    epochs=50,
    dropout=0.0,
    verbose=False,
    global_tracker=None
```

+ Ask Gemini

Flash ▾



Gemini is AI and can make mistakes.